Indian Institute of Technology, Hyderabad

भारतीय प्रौद्योगिकी संस्थान हैदराबाद
Indian Institute of Technology Hyderabad

Course Name

Computer Aided Numerical Methods – I

Numerical Methods for Polynomial Interpolation, Approximation and Root Finding

---

Group 2

Project Documentation

Contents

1 Interpolation Methods

1.1 The Newton Form of the Interpolating Polynomial

Introduction to Interpolation

In many practical problems arising in science and engineering, one is often provided with a finite set of data points obtained either through experimental measurements or from evaluations of a function whose analytical form is unknown or difficult to handle. The problem of interpolation consists of determining a function that exactly passes through these given data points. More specifically, given a set of $n+1$ distinct points $(x_0, y_0), (x_1, y_1), \ldots, (x_n, y_n)$, the objective is to construct a polynomial $P(x)$ of degree at most n such that $P(x_i) = y_i$ for each i. This polynomial is referred to as the *interpolating polynomial*.

Newton Form of the Interpolating Polynomial

The objective of this section is to write $P_{0,1,2,\ldots,n}(x)$, the unique polynomial of degree at most n which interpolates data at the $n+1$ distinct abscissas $x_0, x_1, x_2, \ldots, x_n$, in Newton Form:

$$\begin{aligned} P_{0,1,2,\ldots,n}(x) &= a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots \\ &\quad + a_n(x - x_0)(x - x_1)\cdots(x - x_{n-1}) \\ &= \sum_{k=0}^{n} a_k \left(\prod_{i=0}^{k-1} (x - x_i) \right) \end{aligned}$$

The advantage of the Newton representation is in the computation of the polynomial coefficients $a_0, a_1, a_2, \ldots, a_n$. Determining the coefficients for the standard representation requires the solution of a system of $n+1$ linear equations. The number of operations needed to solve this system is $O(n^3)$. The algorithm to be developed presently for determining the coefficients of the Newton form requires only $O(n^2)$ operations. For even moderate values of n, this represents a significant computational savings.

Divided Differences

To determine the coefficients of the Newton form of the interpolating polynomial, each of the interpolating conditions

$$P_{0,1,\ldots,n}(x_k) = f(x_k)$$

$(k = 0, 1, 2, \ldots, n)$ must be applied. The special structure of the Newton form leads to a system of equations of the form

$$\begin{aligned} a_0 &= f(x_0) \\ a_0 + a_1(x_1 - x_0) &= f(x_1) \\ a_0 + a_1(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1) &= f(x_2) \\ &\vdots \\ a_0 + a_1(x_n - x_0) + a_2(x_n - x_0)(x_n - x_1) + \cdots + a_n(x_n - x_0)\cdots(x_n - x_{n-1}) &= f(x_n). \end{aligned}$$

The solution to this system of equations can be obtained easily by forward substitution and can be expressed compactly in terms of *divided differences*.

Determination of Coefficients Using Divided Differences

The system of equations obtained from the interpolation conditions can be solved sequentially due to its triangular structure. Starting with the first equation,

$$a_0 = f(x_0),$$

we substitute this value into the second equation:

$$a_0 + a_1(x_1 - x_0) = f(x_1).$$

Using $a_0 = f(x_0)$, we obtain

$$a_1 = \frac{f(x_1) - f(x_0)}{x_1 - x_0}.$$

Proceeding similarly, consider the third equation:

$$a_0 + a_1(x_2 - x_0) + a_2(x_2 - x_0)(x_2 - x_1) = f(x_2).$$

Substituting the known values of a_0 and a_1, we solve for a_2:

$$a_2 = \frac{f(x_2) - \left[f(x_0) + a_1(x_2 - x_0)\right]}{(x_2 - x_0)(x_2 - x_1)}.$$

Continuing this process recursively, each coefficient a_k can be expressed in terms of previously computed coefficients.

To simplify notation and computation, the concept of divided differences is introduced. Define

$$f[x_0] = f(x_0),$$

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0},$$

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0}.$$

In general, the divided difference of order k is defined as

$$f[x_0, x_1, \ldots, x_k] = \frac{f[x_1, \ldots, x_k] - f[x_0, \ldots, x_{k-1}]}{x_k - x_0}.$$

Final Expression for the Newton Interpolating Polynomial

With this notation, the coefficients of the Newton interpolating polynomial can be written as

$$a_k = f[x_0, x_1, \ldots, x_k], \quad k = 0, 1, \ldots, n.$$

Thus, the interpolating polynomial takes the form

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots + f[x_0, \ldots, x_n] \prod_{i=0}^{n-1} (x - x_i).$$

Tabular Representation of Divided Differences

For practical computation, the divided differences are arranged in a triangular table. The first column of the table consists of the function values

$$f[x_i] = f(x_i), \quad i = 0, 1, \ldots, n.$$

Each subsequent column contains higher-order divided differences computed recursively.

The entries of the table are obtained using the relation

$$f[x_i, x_{i+1}, \ldots, x_{i+j}] = \frac{f[x_{i+1}, \ldots, x_{i+j}] - f[x_i, \ldots, x_{i+j-1}]}{x_{i+j} - x_i}, \quad j \geq 1.$$

The structure of the divided difference table is triangular and can be represented as follows:

x_i	$f[x_i]$	$f[x_i, x_{i+1}]$	$f[x_i, x_{i+1}, x_{i+2}]$	$\cdots$
x_0	$f[x_0]$	$f[x_0, x_1]$	$f[x_0, x_1, x_2]$	$\cdots$
x_1	$f[x_1]$	$f[x_1, x_2]$	$f[x_1, x_2, x_3]$	$\cdots$
x_2	$f[x_2]$	$f[x_2, x_3]$	$\cdots$	
$\vdots$	$\vdots$			
x_n	$f[x_n]$			

Each column corresponds to divided differences of increasing order, and each entry depends only on two entries from the previous column. This recursive structure allows efficient computation of all coefficients.

Extraction of Polynomial Coefficients

An important feature of the divided difference table is that the coefficients of the Newton interpolating polynomial are given by the first row of the table. Specifically,

$$a_0 = f[x_0], \quad a_1 = f[x_0, x_1], \quad a_2 = f[x_0, x_1, x_2], \quad \ldots, \quad a_n = f[x_0, x_1, \ldots, x_n].$$

Thus, once the divided difference table has been constructed, the interpolating polynomial can be written directly in the Newton form without any further computation.

Algorithm and Implementation of Newton Divided Difference Interpolation

The computational procedure for constructing the Newton interpolating polynomial follows directly from the tabular formulation. Given a set of $n + 1$ data points, a two-dimensional array is initialized to store the divided differences. The input data, consisting of values x_i and $f(x_i)$, are read from an external `.csv` file and stored in suitable data structures.

The first column of the divided difference table is populated with the known function values:

$$\texttt{table}[i][0] = f(x_i), \qquad i = 0, 1, \ldots, n.$$

Higher-order divided differences are then computed sequentially via the recurrence relation

$$\texttt{table}[i][j] = \frac{\texttt{table}[i+1][j-1] - \texttt{table}[i][j-1]}{x_{i+j} - x_i}, \qquad j \geq 1,$$

proceeding column by column until the table is fully populated. The recursive structure ensures that previously computed entries are reused efficiently, reducing the overall computational complexity to $O(n^2)$—considerably lower than the cost of solving an equivalent system of linear equations.

Once the table is complete, the polynomial coefficients are extracted from its first row. These coefficients define the Newton form of the interpolating polynomial, which can be evaluated at any point in the domain. In this implementation, the polynomial was evaluated over a prescribed set of points within the data range, and the resulting values were written to an output file for subsequent visualization and analysis.

Conversion to Standard Polynomial Form

The Newton interpolating polynomial obtained in the previous section is expressed in the form

$$P_n(x) = a_0 + a_1(x - x_0) + a_2(x - x_0)(x - x_1) + \cdots + a_n \prod_{i=0}^{n-1} (x - x_i),$$

For further analysis, particularly for differentiation and root-finding, it is convenient to express the polynomial in the standard power form

$$P_n(x) = c_0 + c_1 x + c_2 x^2 + \cdots + c_n x^n.$$

The conversion is carried out by expanding the product terms iteratively. Let

$$T_0(x) = 1,$$

$$and\ define,\ T_{k+1}(x) = (x - x_k)\, T_k(x).$$

$$Then\ the\ Newton\ polynomial\ can\ be\ written\ as,\ \ P_n(x) = \sum_{k=0}^{n} a_k\, T_k(x).$$

Each polynomial $T_k(x)$ is represented in coefficient form, and the next term is obtained by multiplying the current polynomial by $(x - x_k)$. If

$$T_k(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_k x^k,$$

$$then,\ T_{k+1}(x) = x \cdot T_k(x) - x_k \cdot T_k(x).$$

This leads to a recursive update of coefficients. Multiplication by x shifts the coefficients to higher powers, while multiplication by $-x_k$ adjusts the existing coefficients. Thus, each new polynomial is constructed from the previous one by shifting and scaling its coefficients.

The contributions from each term $a_k T_k(x)$ are accumulated to obtain the coefficients of the polynomial in standard form. This procedure directly mirrors the implementation, where polynomial multiplication is performed using coefficient arrays.

1.2 Forward Difference Method for Equally Spaced Data

In the given dataset, the interpolation nodes are equally spaced, that is,

$$x_i = x_0 + ih, \quad i = 0, 1, \ldots, n,$$

where h is the constant spacing between successive points.

For such uniformly spaced data, the general Newton divided difference formulation can be simplified to obtain a more computationally efficient representation known as the Newton forward difference method. In contrast to the divided difference approach, which involves division by varying denominators, the forward difference method relies only on successive differences and a constant step size. Consequently, the computational effort is reduced and the structure of the interpolation polynomial becomes simpler.

Forward Difference Operator

The first forward difference is given by,

$$\Delta f(x_i) = f(x_{i+1}) - f(x_i).$$

Higher-order forward differences are defined recursively as

$$\Delta^2 f(x_i) = \Delta f(x_{i+1}) - \Delta f(x_i), \quad \Delta^3 f(x_i) = \Delta^2 f(x_{i+1}) - \Delta^2 f(x_i),$$

and in general,

$$\Delta^k f(x_i) = \Delta^{k-1} f(x_{i+1}) - \Delta^{k-1} f(x_i).$$

Newton Forward Interpolation Formula

Consider the Newton interpolating polynomial in its general form:

$$P_n(x) = f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \cdots .$$

If the nodes are equally spaced, we have

$$x_i = x_0 + ih,$$

We now express the divided differences in terms of forward differences.
The first divided difference becomes

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0} = \frac{\Delta f(x_0)}{h}.$$

Similarly, the second divided difference is

$$f[x_0, x_1, x_2] = \frac{f[x_1, x_2] - f[x_0, x_1]}{x_2 - x_0} = \frac{\frac{\Delta f(x_1)}{h} - \frac{\Delta f(x_0)}{h}}{2h} = \frac{\Delta^2 f(x_0)}{2!\, h^2}.$$

Continuing in this manner, the kth divided difference can be written as

$$f[x_0, x_1, \ldots, x_k] = \frac{\Delta^k f(x_0)}{k!\, h^k}.$$

Substituting these expressions into the Newton polynomial, we get

$$P_n(x) = f(x_0) + \frac{\Delta f(x_0)}{h}(x - x_0) + \frac{\Delta^2 f(x_0)}{2!\, h^2}(x - x_0)(x - x_1) + \cdots .$$

$$since,\ \ s = \frac{x - x_0}{h},$$

$$we\ obtain,\ (x - x_0)(x - x_1)\cdots(x - x_{k-1}) = h^k\, s(s-1)\cdots(s-k+1).$$

Substituting into the polynomial, all powers of h cancel, yielding

$$P_n(x) = f(x_0) + s\Delta f(x_0) + \frac{s(s-1)}{2!}\Delta^2 f(x_0) + \frac{s(s-1)(s-2)}{3!}\Delta^3 f(x_0) + \cdots .$$

This is the Newton forward difference interpolation formula.

Forward Difference Table

The forward differences are arranged in a tabular form. The first column consists of the function values, while each subsequent column contains higher-order forward differences computed using successive subtraction.

x_i	$f(x_i)$	$\Delta f(x_i)$	$\Delta^2 f(x_i)$	$\cdots$
x_0	$f(x_0)$	$\Delta f(x_0)$	$\Delta^2 f(x_0)$	$\cdots$
x_1	$f(x_1)$	$\Delta f(x_1)$	$\Delta^2 f(x_1)$	$\cdots$
x_2	$f(x_2)$	$\Delta f(x_2)$	$\cdots$	
$\vdots$	$\vdots$			

The coefficients required in the forward interpolation formula are obtained from the first row of this table.

Implementation of Forward Difference Method

In the present work, the forward difference method was implemented by constructing the forward difference table using a two-dimensional array. The first column was initialized with the given function values, and higher-order differences were computed using successive subtraction.

The interpolating polynomial was evaluated using the forward difference formula in terms of the parameter $s = \frac{x-x_0}{h}$. The factorial terms appearing in the denominator were computed iteratively.

Since the computation involves only subtraction and multiplication by constant factors, the forward difference method is computationally more efficient than the general divided difference method for equally spaced data.

Comparison with Divided Difference Method

In the present work, the interpolating polynomial was constructed using both the Newton divided difference method and the forward difference method.

From the uniqueness theorem, it follows that the interpolating polynomial corresponding to a given set of data points is unique. Hence, both the Newton divided difference method and the forward difference method must yield the same polynomial as shown in Fig. 1, which can also be verified by observing that the coefficients of the standard polynomial obtained from both methods are identical.

Thus, the difference between the two approaches lies not in the final polynomial, but in the computational procedure used to obtain it. In particular, when the data points are equally spaced, the forward difference method offers a more efficient and structured approach, as it avoids repeated division operations and relies only on successive differences.

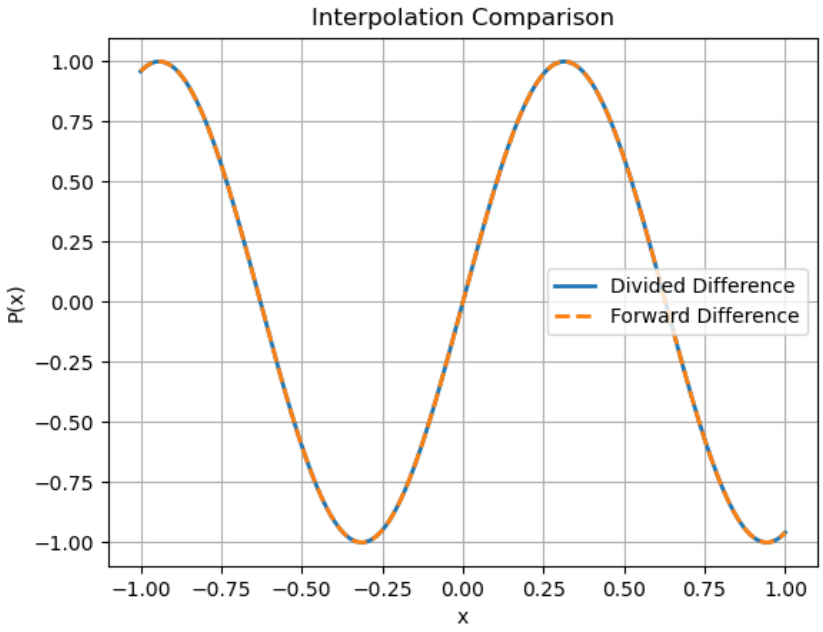

Figure 1: Newton Divided Difference vs Forward Difference

1.3 Error Analysis and Runge's Phenomenon

Error in Polynomial Interpolation

To derive an expression for the error in polynomial interpolation, let $P_{0,1,\ldots,n}$ be the polynomial that interpolates the function f at the points $x_0, x_1, \ldots, x_n$. Further, let P^* denote the polynomial that interpolates f at the points $x_0, x_1, \ldots, x_n$ and an additional point t.

From the Newton form of the interpolating polynomial, we can write

$$P^*(x) = P_{0,1,\ldots,n}(x) + f[x_0, x_1, \ldots, x_n, t] \prod_{i=0}^{n} (x - x_i).$$

Since P^* interpolates f at the point t, it follows that

$$P^*(t) = f(t).$$

Evaluating the above expression at $x = t$, we obtain

$$f(t) = P_{0,1,\ldots,n}(t) + f[x_0, x_1, \ldots, x_n, t] \prod_{i=0}^{n} (t - x_i).$$

Rewriting this expression gives

$$f(t) - P_{0,1,\ldots,n}(t) = f[x_0, x_1, \ldots, x_n, t] \prod_{i=0}^{n} (t - x_i).$$

Since t is arbitrary, we replace t by x to obtain

$$f(x) - P_{0,1,\ldots,n}(x) = f[x_0, x_1, \ldots, x_n, x] \prod_{i=0}^{n} (x - x_i).$$

To relate this divided difference to derivatives, we consider the first-order divided difference

$$f[x_0, x_1] = \frac{f(x_1) - f(x_0)}{x_1 - x_0}.$$

If f is continuous on $[x_0, x_1]$ and differentiable on (x_0, x_1), the Mean Value Theorem ensures that there exists a point $\xi \in (x_0, x_1)$, such that,

$$f[x_0, x_1] = f'(\xi).$$

Extending this idea for sufficiently smooth functions, the divided difference of order $n + 1$ can be written as:

$$f[x_0, x_1, \ldots, x_n, x] = \frac{f^{(n+1)}(\xi)}{(n+1)!}$$

Substituting this into the error expression, we obtain

$$f(x) - P_{0,1,\ldots,n}(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

This gives the required expression for the interpolation error.

Runge's Phenomenon

From the error expression for polynomial interpolation, we have

$$f(x) - P_{0,1,\ldots,n}(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

Assuming that f is sufficiently smooth on $[a, b]$, there exists a constant $M > 0$ such that

$$|f^{(n+1)}(x)| \leq M, \quad \forall x \in [a, b].$$

Thus, the error is bounded by

$$|f(x) - P_{0,1,\ldots,n}(x)| \leq \frac{M}{(n+1)!} \left| \prod_{i=0}^{n} (x - x_i) \right|.$$

Hence, the dominant factor governing the error is the nodal polynomial

$$\omega_{n+1}(x) = \prod_{i=0}^{n} (x - x_i).$$

Now consider the case where the interpolation nodes are equally spaced:

$$x_i = a + ih, \quad h = \frac{b-a}{n}.$$

In this case, the nodes are uniformly distributed across the interval, including near the endpoints. However, this uniform spacing leads to a critical issue: near the endpoints of the interval, the point x is relatively far from most of the nodes.

As a result, many of the factors $(x - x_i)$ in the product $\omega_{n+1}(x)$ become large simultaneously when x is close to a or b. Since the nodal polynomial is a product of $(n + 1)$ such terms, even moderate growth in individual factors leads to a rapid increase in the magnitude of $\omega_{n+1}(x)$ near the endpoints.

Moreover, as n increases, the number of such factors increases, amplifying this effect. Therefore, for equally spaced nodes,

$$\max_{x \in [a,b]} |\omega_{n+1}(x)|, \ \textit{grows rapidly with 'n', especially near the endpoints.}$$

Although the term $\frac{1}{(n+1)!}$ decreases with increasing n, this decrease is insufficient to counterbalance the rapid growth of $\omega_{n+1}(x)$. Consequently, the interpolation error becomes large near the endpoints.

This phenomenon, where polynomial interpolation at equally spaced nodes produces large oscillations near the endpoints despite increasing degree, is known as *Runge's phenomenon*.

Thus, the root cause of Runge's phenomenon lies in the use of equally spaced nodes, which causes the nodal polynomial to grow rapidly near the endpoints due to the simultaneous enlargement of multiple factors in $\omega_{n+1}(x)$.

Observation in the Present Case

Although equally spaced nodes are used, Runge's phenomenon is not observed in our case (group2 dataset) (It can be observed from Fig. 1 that $f(x) = \sin(5x)$).

This can be explained by examining the structure of the interpolation error:

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

The occurrence of Runge's phenomenon depends on the interplay between the two factors in this expression.

For $f(x) = \sin(5x)$, all higher derivatives remain bounded and oscillatory:

$$f^{(k)}(x) = 5^k \sin\left(5x + \phi\right), \ \phi \ is \ phase \ shift \ accounting \ for \ the \ alternating \ sine, \ cosine \ and \ sign.$$

which implies that although the magnitude grows as 5^k, it is controlled and does not grow excessively relative to $(n+1)!$.

As a result, the interpolation error remains small throughout the interval and no visible oscillation occurs near the endpoints.

To illustrate Runge's phenomenon, the group1 dataset is used for constructing the Newton divided difference interpolating polynomial. The resulting plot (Fig. 2) exhibits pronounced oscillations near the endpoints, clearly demonstrating Runge's phenomenon.

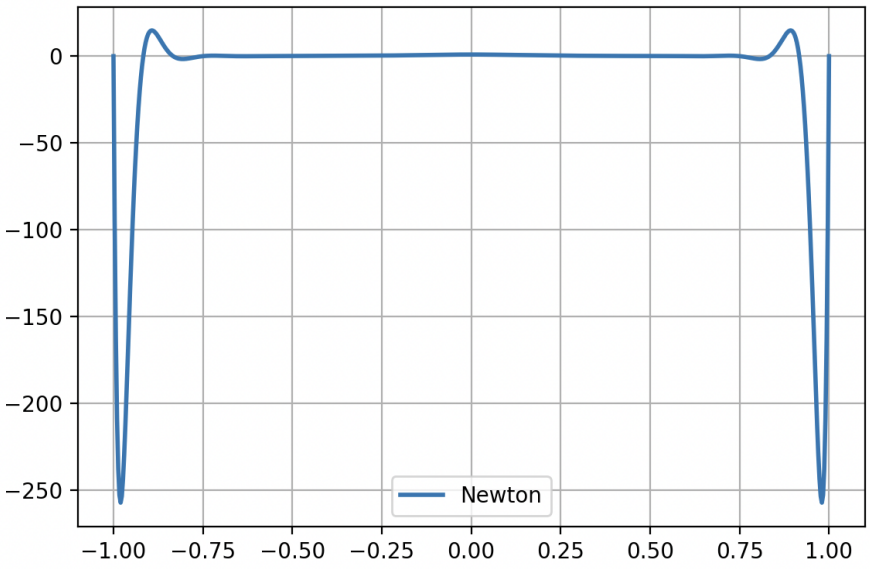

Figure 2: Runge's Phenomenon using group1 dataset

Motivation for Chebyshev

From the error expression, it is evident that for equally spaced nodes, the interpolation error is strongly influenced by the growth of the nodal polynomial

$$\omega_{n+1}(x) = \prod_{i=0}^{n}(x - x_i).$$

Since the nodes in the present dataset are fixed and equally spaced, their distribution cannot be altered to control this growth which may lead to oscillations near the endpoints, as seen in Runge's phenomenon.

To address this issue, we consider an approximation approach instead of exact interpolation. In this approach, a polynomial is constructed to best fit the data in a least squares sense, rather than passing through all points exactly.

A suitable choice for such approximations is the Chebyshev polynomial basis, which has favorable numerical properties and exhibits near-minimal deviation over the interval.

Thus, even with equally spaced nodes, using Chebyshev polynomials within a least squares framework yields stable approximations and avoids the large oscillations.

2 Polynomial Approximation using Chebyshev Basis

2.1 Chebyshev Polynomial Approximation Method

Limitations of High-Degree Polynomial Interpolation

Given $N = 25$ data points, one possible approach is to construct an interpolating polynomial of degree 24 that passes exactly through all points.

For equally spaced nodes, high-degree interpolation is highly unstable and leads to large oscillations near the endpoints of the interval, a behavior known as Runge's phenomenon.

It is important to note that this issue is not resolved by simply changing the polynomial basis. Even if Chebyshev polynomials are used as a basis, constructing a degree 24 interpolating polynomial results in the same unique polynomial (uniqueness theorem) and hence the oscillatory behavior persists.

This shows that Runge's phenomenon is primarily a consequence of using equally spaced nodes together with high-degree interpolation, rather than the choice of basis.

Furthermore, constructing such a polynomial involves solving a 25×25 system, which is computationally expensive and sensitive to numerical errors. Therefore, high-degree interpolation is not a reliable approach for approximating the given dataset.

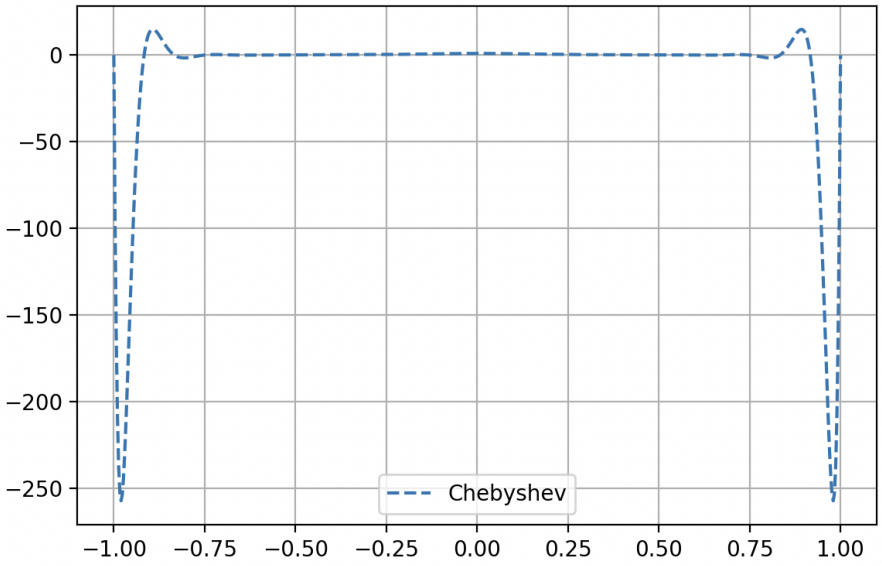

Figure 3: Polynomial interpolation of group1 dataset using Chebyshev basis

Limitations of the Standard Monomial Basis

If the polynomial is expressed in the standard monomial basis,

$$P(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_d x^d,$$

the resulting system involves a Vandermonde-type matrix.

Such matrices are highly ill-conditioned. The reason is that higher powers of x grow rapidly in magnitude, causing the columns of the matrix to become nearly linearly dependent. In finite precision arithmetic, terms like x^{23} and x^{24} become numerically indistinguishable, leading to loss of significance.

Even when the degree of the polynomial is reduced (for example, $d \approx 9$ or 10), the monomial basis remains problematic. Over a bounded interval such as $[-1, 1]$, higher powers of x tend to exhibit similar behavior, especially near the endpoints. This causes the columns corresponding to x^k to become nearly linearly dependent, resulting in poor conditioning of the matrix.

As a consequence, small perturbations in the data can produce large variations in the computed coefficients. Thus, reducing the degree alone is not sufficient to ensure numerical stability and the choice of basis functions plays a crucial role.

Therefore, the standard monomial basis is not suitable for stable polynomial approximation, even when moderate degrees are used.

Choice of Chebyshev Basis

In order to overcome the limitations associated with the standard monomial basis, we replace the monomial basis $\{1, x, x^2, \dots\}$ with Chebyshev polynomials as the basis functions for approximation.

Thus, instead of representing the polynomial in the form

$$P(x) = a_0 + a_1 x + a_2 x^2 + \cdots,$$

$$we\ express\ it\ as,\ P_d(x) = \sum_{k=0}^{d} c_k T_k(x),$$

where $T_k(x)$ are the Chebyshev polynomials and $d \ll N$.

The Chebyshev polynomials are defined by the recurrence relation:

$$T_0(x) = 1, \quad T_1(x) = x,$$

$$T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x).$$

Unlike the standard monomial basis, Chebyshev polynomials remain bounded in magnitude over the interval $[-1, 1]$, satisfying $|T_k(x)| \leq 1$ for all k. This means that the values of the Chebyshev polynomials stay within a fixed range on $[-1, 1]$. As a result, no term becomes too large or too small and all parts of the polynomial contribute in a balanced manner.

In contrast, higher powers of x in the monomial basis tend to either shrink rapidly (near $x = 0$) or become nearly indistinguishable from each other (near $x = 1$) leading to loss of numerical precision. By avoiding such extreme variations, Chebyshev polynomials maintain a consistent scale across the domain, which improves numerical stability.

Additionally, Chebyshev polynomials exhibit near-orthogonality over the interval $[-1, 1]$. This means that different basis functions are largely independent of one another, and their contributions to the approximation do not overlap significantly. As a result, the columns of the corresponding system matrix are well-separated, leading to a better-conditioned system.

Consequently, the matrix arising in the least squares formulation is significantly more stable and less sensitive to numerical errors compared to the monomial basis.

Since Chebyshev polynomials are defined on $[-1, 1]$, a domain transformation is necessary when the given data lies outside this interval. The data points are then scaled to $[-1, 1]$ before applying the Chebyshev basis.

Motivation for Least Squares Approximation

To overcome these issues, instead of constructing a polynomial that exactly interpolates all 25 points, we consider a polynomial of lower degree d, where $d \ll N$.

In this case, the system becomes overdetermined, as we have 25 equations but only $d+1$ unknown coefficients. Therefore, an exact solution satisfying all equations does not exist.

Instead, we determine the coefficients by minimizing the sum of squared errors:

$$\min \sum_{i=1}^{25} \left(y_i - P_d(x_i)\right)^2.$$

This approach ensures that the polynomial captures the overall trend of the data without introducing large oscillations, leading to a more stable and reliable approximation.

Derivation of the Normal Equations from Least Squares

To determine the coefficients $c_0, c_1, \ldots, c_n$, we construct a polynomial approximation of the form

$$p(x) = \sum_{j=0}^{n} c_j T_j(x),$$

where $T_j(x)$ are the Chebyshev polynomials.

For each data point (x_i, y_i), the approximation error is given by

$$E_i = y_i - p(x_i) = y_i - \sum_{j=0}^{n} c_j T_j(x_i).$$

The least squares method minimizes the sum of squared errors:

$$S(c_0, c_1, \ldots, c_n) = \sum_{i=1}^{25} \left(y_i - \sum_{j=0}^{n} c_j T_j(x_i) \right)^2.$$

To minimize S, we differentiate with respect to c_k:

$$\frac{\partial S}{\partial c_k} = \sum_{i=1}^{25} 2 \left(y_i - \sum_{j=0}^{n} c_j T_j(x_i) \right) \cdot \frac{\partial}{\partial c_k} \left(y_i - \sum_{j=0}^{n} c_j T_j(x_i) \right).$$

Thus,

$$\frac{\partial S}{\partial c_k} = -2 \sum_{i=1}^{25} \left(y_i - \sum_{j=0}^{n} c_j T_j(x_i) \right) T_k(x_i).$$

For minimization, we set:

$$\frac{\partial S}{\partial c_k} = 0,$$

which gives:

$$\sum_{i=1}^{25} \left(y_i - \sum_{j=0}^{n} c_j T_j(x_i) \right) T_k(x_i) = 0.$$

Expanding,

$$\sum_{i=1}^{25} y_i T_k(x_i) - \sum_{i=1}^{25} \left(\sum_{j=0}^{n} c_j T_j(x_i) \right) T_k(x_i) = 0.$$

Rearranging,

$$\sum_{j=0}^{n} c_j \left(\sum_{i=1}^{25} T_j(x_i) T_k(x_i) \right) = \sum_{i=1}^{25} y_i T_k(x_i).$$

Matrix Formulation

Define the matrix A as:

$$A_{ik} = T_k(x_i).$$

$$Then, \ \sum_{i=1}^{25} T_j(x_i) T_k(x_i)$$

represents the dot product of column j and column k of A, which forms the matrix $A^T A$.

$$Similarly, \ \sum_{i=1}^{25} y_i T_k(x_i)$$

represents the dot product of column k of A with the vector $\mathbf{y}$, forming $A^T \mathbf{y}$.

Thus, the system can be written as:

$$(A^T A)\mathbf{c} = A^T \mathbf{y}.$$

Numerical Advantage of Chebyshev Basis

In the standard monomial basis, the corresponding terms involve powers of x:

$$\sum_{i=1}^{25} x_i^j x_i^k,$$

which become nearly linearly dependent for large j and k, resulting in an ill-conditioned matrix.

In contrast, Chebyshev polynomials satisfy near-orthogonality, meaning that:

$$\sum_{i=1}^{25} T_j(x_i) T_k(x_i) \approx 0 \quad \text{for } j \neq k.$$

Thus, the matrix $A^T A$ becomes close to diagonal, significantly improving numerical stability and ensuring that the least squares solution can be computed accurately.

Algorithmic Implementation of the Matrix Method

The derived normal equation

$$(A^T A)\mathbf{c} = A^T \mathbf{y}$$

is implemented computationally using matrix operations.

Step 1: Construction of the Design Matrix

The matrix A is constructed such that:

$$A_{ik} = T_k(x_i),$$

where $T_k(x)$ are the Chebyshev polynomials.

In the implementation, each entry is computed using the recurrence relation for Chebyshev polynomials. Thus, each row corresponds to a data point, and each column corresponds to a basis function.

Step 2: Computation of $A^T A$

The matrix $A^T A$ is computed using the relation:

$$(A^T A)_{jk} = \sum_{i=1}^{25} A_{ij} A_{ik}.$$

Step 3: Computation of $A^T \mathbf{y}$

The vector $A^T \mathbf{y}$ is computed as:

$$(A^T \mathbf{y})_k = \sum_{i=1}^{25} A_{ik} y_i.$$

This corresponds to taking the dot product of each column of A with the data vector $\mathbf{y}$.

Step 4: Solution of the Linear System

The resulting system

$$(A^T A)\mathbf{c} = A^T \mathbf{y}$$

is solved using Gaussian elimination with partial pivoting.

Partial pivoting ensures numerical stability by selecting the largest pivot element at each step, thereby reducing the effect of round-off errors.

Step 5: Interpretation of the Solution

The computed vector $\mathbf{c}$ contains the coefficients of the Chebyshev polynomial approximation:

$$P(x) = \sum_{k=0}^{n} c_k T_k(x).$$

These coefficients are then used for further analysis, including conversion to standard polynomial form and evaluation of approximation error.

Gradient Descent Method for Least Squares Approximation

In addition to solving the normal equations directly, the least squares problem can also be approached as an optimization problem. Instead of solving

$$(A^T A)\mathbf{c} = A^T \mathbf{y},$$

we minimize the error function iteratively using gradient descent.

Step 1: Objective Function

The mean squared error is defined as:

$$J(\mathbf{c}) = \frac{1}{2N} \sum_{i=1}^{N} (P(x_i) - y_i)^2,$$

$$where, \;\; P(x_i) = \sum_{k=0}^{n} c_k T_k(x_i).$$

Step 2: Residual Formulation

For each data point, define the residual:

$$r_i = P(x_i) - y_i.$$

The objective function can then be written as:

$$J(\mathbf{c}) = \frac{1}{2N} \sum_{i=1}^{N} r_i^2.$$

Step 3: Derivation of the Gradient

To minimize J, we compute the partial derivative with respect to c_k:

$$\frac{\partial J}{\partial c_k} = \frac{1}{N} \sum_{i=1}^{N} r_i \frac{\partial P(x_i)}{\partial c_k}.$$

$$Since, \;\; P(x_i) = \sum_{j=0}^{n} c_j T_j(x_i),$$

$$we\ obtain: \;\; \frac{\partial P(x_i)}{\partial c_k} = T_k(x_i).$$

$$Thus, \;\; \frac{\partial J}{\partial c_k} = \frac{1}{N} \sum_{i=1}^{N} r_i T_k(x_i).$$

In matrix form, this can be written as:

$$\nabla J = \frac{1}{N} A^T (A\mathbf{c} - \mathbf{y}),$$

which shows that gradient descent operates on the same structure as the normal equations.

Step 4: Iterative Update Rule

The coefficients are updated iteratively using:

$$c_k^{(new)} = c_k^{(old)} - \alpha \frac{\partial J}{\partial c_k}, \; where\ \alpha\ is\ the\ learning\ rate.$$

This update moves the solution in the direction opposite to the gradient, thereby reducing the error at each iteration.

Step 5: Algorithmic Implementation

In the implementation, the gradient descent algorithm begins by initializing the coefficient vector $\mathbf{c}$ to zero and constructing the design matrix A, where each entry is computed as $A_{ik} = T_k(x_i)$ using the Chebyshev recurrence.

At each iteration, the polynomial values $P(x_i)$ are evaluated by computing the matrix-vector product $A\mathbf{c}$, following which the residuals are obtained as $r_i = P(x_i) - y_i$.

The gradient is then computed using the relation $\nabla J = \frac{1}{N} A^T \mathbf{r}$, which is implemented through nested loops that accumulate the products of residuals and corresponding basis function values. The coefficients are updated using the rule $\mathbf{c} := \mathbf{c} - \alpha \nabla J$, where α is the learning rate.

Step 6: Convergence Criterion

The iterations are terminated when the magnitude of the gradient becomes sufficiently small:

$$\|\nabla J\| < \epsilon.$$

This indicates that the solution has reached a local minimum of the error function.

Step 7: Error Evaluation

Upon convergence, the accuracy of the approximation is quantified using two metrics:

- **Maximum Absolute Error (E_{max}):** Captures the worst-case deviation, defined as $\max |P(x_i) - y_i|$.
- **Root Mean Square Error (E_{rms}):** Measures the average deviation, defined as $\sqrt{\frac{1}{N} \sum (P(x_i) - y_i)^2}$.

These measures provide quantitative insight into the quality of the fitted polynomial.

Significance of the Method

Gradient descent provides an iterative alternative to direct matrix-based methods and avoids explicit matrix inversion. It is particularly useful for large-scale problems and offers insight into the convergence behavior of the solution. In the present case, it is used as an independent approach to verify the results obtained from the matrix formulation.

QR Factorization Approach

Although the least squares problem can be solved using the normal equations $(A^T A)\mathbf{c} = A^T \mathbf{y}$, this approach suffers from numerical instability. The primary difficulty arises because forming the matrix $A^T A$ squares the condition number of A, thereby amplifying any numerical errors present in the data or introduced during computation. Consequently, even small perturbations can lead to large variations in the computed coefficients, making the solution unreliable.

To avoid this issue, QR factorization is used as a more stable alternative. In this approach, the matrix A is decomposed as

$$A = QR,$$

where Q is an orthogonal matrix satisfying $Q^T Q = I$, and R is an upper triangular matrix.

Substituting this decomposition into the least squares problem

$$\min \|A\mathbf{c} - \mathbf{y}\|^2,$$

$$we\ obtain,\ \ \min \|QR\mathbf{c} - \mathbf{y}\|^2.$$

Multiplying both sides by Q^T, and using the fact that orthogonal transformations preserve vector norms, the problem reduces to

$$\min \|R\mathbf{c} - Q^T \mathbf{y}\|^2.$$

The vector $Q^T \mathbf{y}$ represents the coordinates of $\mathbf{y}$ in the orthonormal basis formed by the columns of Q. This transformation separates $\mathbf{y}$ into two components: one that lies within the column space of A, and another that is orthogonal to it. Accordingly, we can write

$$Q^T \mathbf{y} = \begin{bmatrix} \mathbf{y}_1 \\ \mathbf{y}_2 \end{bmatrix},$$

where $\mathbf{y}_1$ corresponds to the components along the first $d+1$ columns of Q (which span the column space of A), and $\mathbf{y}_2$ corresponds to the remaining orthogonal directions.

Since any vector of the form $A\mathbf{c}$ lies entirely within the column space of A, it cannot have any component in the orthogonal directions. Therefore, the component $\mathbf{y}_2$ cannot be matched by $A\mathbf{c}$ and contributes directly to the residual error.

Thus, the least squares problem reduces to minimizing

$$\|R\mathbf{c} - \mathbf{y}_1\|^2 + \|\mathbf{y}_2\|^2.$$

Since $\mathbf{y}_2$ is independent of $\mathbf{c}$, it cannot be reduced. Therefore, the minimization problem simplifies to solving

$$R\mathbf{c} = \mathbf{y}_1.$$

Because R is an upper triangular matrix, this system can be solved efficiently using back substitution. The resulting solution $\mathbf{c}$ ensures that the component of $\mathbf{y}$ lying within the column space of A is matched as closely as possible, while the remaining orthogonal component represents the minimum achievable error.

In this way, QR factorization directly minimizes the least squares error without forming A^TA, thereby avoiding the amplification of numerical errors. The use of orthogonal transformations ensures that the computation remains stable, and the resulting solution is both accurate and reliable.

Implementation of QR Factorization using Householder Transformations

In the present work, the QR factorization of the matrix A is computed using Householder transformations. This approach is preferred over classical methods such as Gram-Schmidt due to its superior numerical stability and its ability to maintain orthogonality in finite precision arithmetic.

The central idea of the Householder method is to construct a sequence of orthogonal transformations that progressively convert the matrix A into an upper triangular matrix R. Each transformation is designed to eliminate the entries below the diagonal in a given column.

Let $\mathbf{x}$ denote the subvector consisting of the entries of the k-th column of A, starting from the diagonal element A_{kk}:

$$\mathbf{x} = \begin{bmatrix} A_{kk} \\ A_{k+1,k} \\ \vdots \\ A_{m,k} \end{bmatrix}.$$

The objective is to transform this vector into a multiple of the first coordinate vector, that is,

$$H\mathbf{x} = \begin{bmatrix} \alpha \\ 0 \\ \vdots \\ 0 \end{bmatrix}, \quad where\ \alpha = \pm\|\mathbf{x}\|.$$

To achieve this, a vector $\mathbf{v}$ is constructed as

$$\mathbf{v} = \mathbf{x} - \alpha\mathbf{e}_1,$$

where $\mathbf{e}_1$ is the first standard basis vector. Using this vector, the Householder transformation is defined as

$$H = I - \frac{2\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}}.$$

This transformation is orthogonal and reflects vectors about the hyperplane perpendicular to $\mathbf{v}$.

Applying H to the matrix A eliminates all entries below the diagonal in the k-th column. This process is repeated for each column, resulting in a sequence of transformations:

$$R = H_n H_{n-1} \cdots H_1 A.$$

In practice, the matrix Q is not formed explicitly. Instead, the same sequence of transformations is applied directly to the vector $\mathbf{y}$ to compute

$$Q^T \mathbf{y} = H_n H_{n-1} \cdots H_1 \mathbf{y}.$$

This avoids unnecessary computation and improves numerical efficiency.

The implementation follows these steps:

1. For each column k, compute the norm of the subvector $\mathbf{x}$.
2. Choose $\alpha = \pm\|\mathbf{x}\|$ to avoid numerical cancellation.
3. Construct the Householder vector $\mathbf{v} = \mathbf{x} - \alpha \mathbf{e}_1$.
4. Apply the transformation to all remaining columns of A using

$$A \leftarrow A - \frac{2\mathbf{v}(\mathbf{v}^T A)}{\mathbf{v}^T \mathbf{v}}.$$

5. Apply the same transformation to the vector $\mathbf{y}$ to update it to $Q^T \mathbf{y}$.
6. Repeat the process for all columns to obtain the upper triangular matrix R.

After the completion of these steps, the matrix A is transformed into R, and the vector $\mathbf{y}$ is transformed into $Q^T \mathbf{y}$. The least squares solution is then obtained by solving the system

$$R\mathbf{c} = \mathbf{y}_1,$$

where $\mathbf{y}_1$ consists of the first $d + 1$ entries of $Q^T \mathbf{y}$.

This approach avoids the explicit formation of the matrix Q, reduces computational complexity and ensures high numerical stability. As a result, the Householder QR method provides a reliable and efficient way to solve the least squares problem for polynomial approximation.

2.2 Error Analysis in Least Squares Approximation

In the least squares approach, the polynomial $P_d(x)$ does not necessarily interpolate all the given data points exactly. Instead, it is chosen such that it minimizes the total squared error over the dataset.

The error at each data point is defined as:

$$r_i = y_i - P_d(x_i),$$

and the total error is given by:

$$S = \sum_{i=1}^{N} r_i^2 = \sum_{i=1}^{N} (y_i - P_d(x_i))^2.$$

Unlike interpolation methods, where the error is zero at the nodes, the least squares method distributes the error across all data points in an optimal manner.

Orthogonality Condition of the Error

A fundamental property of the least squares solution is that the residual vector is orthogonal to the space spanned by the basis functions.

From the normal equations:

$$(A^T A)\mathbf{c} = A^T \mathbf{y},$$

we obtain:

$$A^T(A\mathbf{c} - \mathbf{y}) = 0.$$

Let $\mathbf{r} = A\mathbf{c} - \mathbf{y}$. Then:

$$A^T \mathbf{r} = 0.$$

This condition can also be interpreted through the QR factorization. Since $A = QR$, we have

$$A\mathbf{c} = QR\mathbf{c},$$

and the residual becomes

$$\mathbf{r} = QR\mathbf{c} - \mathbf{y}.$$

Multiplying by Q^T, we obtain

$$Q^T\mathbf{r} = R\mathbf{c} - Q^T\mathbf{y}.$$

At the solution, $R\mathbf{c} = \mathbf{y}_1$, which implies that the first $d+1$ components of $Q^T\mathbf{r}$ are zero. Thus, the residual lies entirely in the orthogonal complement of the column space, confirming that it is orthogonal to all basis functions.

This implies:

$$\sum_{i=1}^{N} r_i T_k(x_i) = 0, \quad \text{for } k = 0, 1, \ldots, d.$$

Thus, the error is orthogonal to each basis function. This ensures that no further reduction in error is possible within the chosen polynomial space.

Comparision of Least Squares method

Let us compare the three approximation methods by looking at RMS error at the 25 given data points for each method.

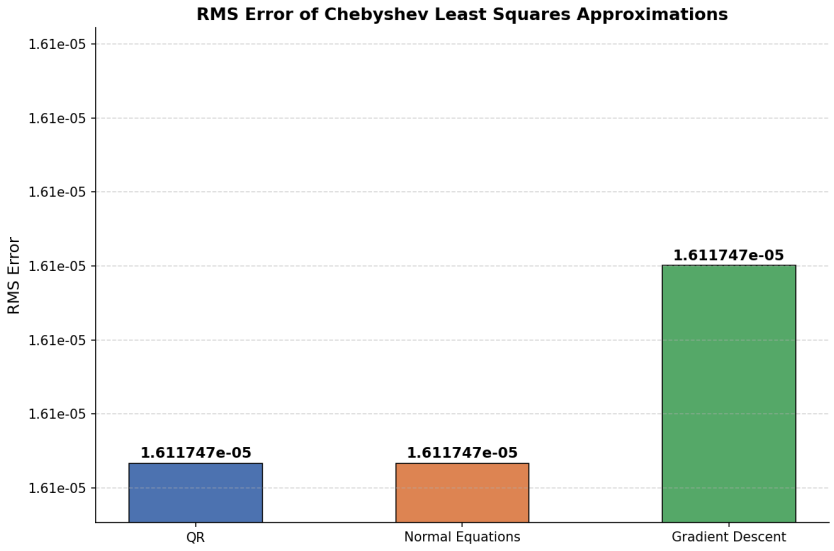

Figure 4: RMS error analysis.

- **Direct vs. Iterative Solvers:** The RMS error comparison demonstrates that **QR Factorization** and the **Normal Equations** both slightly outperform **Gradient Descent**. This is because Gradient Descent is an iterative method that requires precise tuning of learning rates and can stall before reaching the global minimum. In contrast, direct solvers like QR and Normal Equations provide an analytical solution that hits the mathematical minimum in a single operation, resulting in a better polynomial approximation.
- **Numerical Stability and Condition Numbers:** While the Normal Equations and QR Factorization show identical accuracy in this specific test, we have selected **QR Factorization** as the definitive method for the next stages. The Normal Equations **squares the condition number** of the system matrix. For high-degree polynomials, this squaring may lead to numerical instability and significant precision loss.
- **The Advantage of QR Decomposition:** By using QR decomposition ($A = QR$), we solve the least-squares problem using an orthogonal basis (Q) and an upper triangular matrix (R). This allows us to maintain the original condition number of the matrix. This ensures that our polynomial approximation remains accurate, even as we transition to more complex data or higher-degree approximations where the system becomes "ill-conditioned."

Comparison with Interpolation Error

In interpolation methods such as Newton divided difference, the error is given by:

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi)}{(n+1)!} \prod_{i=0}^{n} (x - x_i).$$

This expression shows that the error depends strongly on the nodal polynomial and can grow rapidly for equally spaced nodes, leading to Runge's phenomenon.

In contrast, the least squares method avoids this issue by not enforcing exact agreement at every point. Instead, it minimizes the overall error, resulting in a smoother and more stable approximation.

3 Root Finding

3.1 Root Finding Using Newton-Raphson Method

Problem Formulation

In the present work, the function is not given explicitly but is obtained through polynomial approximation methods such as Newton divided difference and Chebyshev least squares approximation. The resulting polynomial is expressed in standard form:

$$P(x) = \sum_{i=0}^{n} a_i x^i.$$

The objective is to determine the roots of the derivative of the approximated polynomial by solving :

$$P'(x) = 0.$$

Derivation of Newton-Raphson Method

Let x_k be an approximation to a root of the equation $f(x) = 0$. To improve this approximation, we expand $f(x)$ about x_k using Taylor's series:

$$f(x) = f(x_k) + (x - x_k)f'(x_k) + \frac{(x - x_k)^2}{2!} f''(x_k) + \cdots$$

If x_k is sufficiently close to the root, the higher-order terms become very small and can be neglected. Thus, we approximate:

$$f(x) \approx f(x_k) + (x - x_k)f'(x_k).$$

To obtain a better approximation to the root, we set $f(x) = 0$:

$$0 = f(x_k) + (x - x_k)f'(x_k).$$

Rearranging the above equation:

$$(x - x_k)f'(x_k) = -f(x_k),$$

$$x - x_k = -\frac{f(x_k)}{f'(x_k)}.$$

Hence, the improved approximation is given by:

$$x = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Replacing x by the next iterate x_{k+1}, we obtain the iterative formula:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Algorithmic Implementation

In this project, we take:

$$f(x) = P'(x),$$

which implies:

$$f'(x) = P''(x).$$

Since the polynomial is available in standard form, its derivatives can be computed directly as:

$$P'(x) = \sum_{i=1}^{n} i a_i x^{i-1},$$

$$P''(x) = \sum_{i=2}^{n} i(i-1) a_i x^{i-2}.$$

Substituting into the Newton-Raphson formula, we obtain:

$$x_{k+1} = x_k - \frac{P'(x_k)}{P''(x_k)}.$$

The implementation performs root finding on the derivative polynomial $P'(x)$ using iterative methods. The coefficients of the derivative $P'(x)$ and the second derivative $P''(x)$ are obtained analytically from the given polynomial coefficients, thereby avoiding numerical differentiation.

The domain $[a, b]$ is determined from the given data points and is divided into a large number of small subintervals. On each subinterval $[x_1, x_2]$, the values of $P'(x)$ are evaluated at the endpoints. If a sign change occurs, that is, if $P'(x_1)\,P'(x_2) \leq 0$, a root is assumed to lie within that interval. This provides a set of initial guesses for the iterative methods.

For the Newton–Raphson method, the initial guess is taken as the midpoint of the interval. The iteration then proceeds using

$$x_{k+1} = x_k - \frac{P'(x_k)}{P''(x_k)},$$

where both $P'(x_k)$ and $P''(x_k)$ are evaluated using the polynomial coefficients. The process is repeated until the difference between successive iterates satisfies a prescribed tolerance.

$$|x_{k+1} - x_k| < \varepsilon$$

Newton Method Roots (Newton Polynomial)

Root No.	Calculated Value
1	−0.9424777960768031
2	−0.3141592653589793
3	0.3141592653589793
4	0.9424777960750381

Newton Method Roots (Chebyshev Polynomial)

Root No.	Calculated Value
1	−0.9424823149026370
2	−0.3141591286956197
3	0.3141591286956197
4	0.9424823149026374

3.2 Root Finding Using Secant Method

The Secant method is an iterative technique used to approximate the roots of a nonlinear equation without requiring the evaluation of derivatives. In the present implementation, it is applied to the derivative polynomial $P'(x)$ to determine its roots.

Given two initial approximations x_0 and x_1, the method constructs a secant line through the points $(x_0, P'(x_0))$ and $(x_1, P'(x_1))$, and uses the intersection of this line with the x-axis as the next approximation. The iteration is given by

$$x_{k+1} = x_k - P'(x_k)\,\frac{x_k - x_{k-1}}{P'(x_k) - P'(x_{k-1})}.$$

The Secant method can be interpreted as a derivative-free approximation of the Newton–Raphson method. The Newton–Raphson iteration is given by

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

In the Secant method, the derivative $f'(x_k)$ is not computed explicitly. Instead, it is approximated using a finite difference based on two successive iterates:

$$f'(x_k) \approx \frac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}.$$

Substituting this approximation into the Newton–Raphson formula gives

$$x_{k+1} = x_k - \frac{f(x_k)}{\dfrac{f(x_k) - f(x_{k-1})}{x_k - x_{k-1}}} = x_k - f(x_k)\,\frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})},$$

which is precisely the Secant iteration.

Thus, the Secant method avoids the explicit evaluation of derivatives while retaining a similar update structure to the Newton–Raphson method. However, since the derivative is only approximated, the convergence is generally slower. In particular, the Newton–Raphson method exhibits quadratic convergence, whereas the Secant method exhibits superlinear convergence of order approximately 1.618.

Algorithmic Implementation

In the implementation, the initial guesses are taken as the endpoints of subintervals where a sign change in $P'(x)$ is detected. This ensures that the root lies within the chosen interval and provides a reliable starting point for the iteration.

The process is repeated until convergence is achieved, which is determined by the condition

$$|x_{k+1} - x_k| < \varepsilon,$$

where ε is a prescribed tolerance. A safeguard is also included to terminate the iteration if the denominator becomes too small, thereby avoiding numerical instability.

Since the Secant method does not require the evaluation of the second derivative $P''(x)$, it is computationally less expensive than the Newton–Raphson method. However, its convergence is generally slower and depends more strongly on the choice of initial guesses.

After computing a root, it is checked to ensure that it lies within the domain $[a, b]$. Duplicate roots arising from adjacent intervals are removed by comparing the distance between computed roots within a specified tolerance.

Secant Method Roots (Newton Polynomial)

Root No.	Calculated Value
1	−0.9424777960768031
2	−0.3141592653589793
3	0.3141592653589793
4	0.9424777960750381

Secant Method Roots (Chebyshev Polynomial)

Root No.	Calculated Value
1	−0.9424823149026370
2	−0.3141591286956197
3	0.3141591286956197
4	0.9424823149026374

4 Comparison of Interpolation Methods and Observations

4.1 Overview of Methods

In the present work, polynomial approximation and interpolation were carried out using multiple approaches, namely:

- Newton divided difference interpolation
- Newton forward difference interpolation (for equally spaced data)
- Least squares approximation using Chebyshev basis
- Numerical solution of least squares using:
  - Normal equations
  - Gradient descent
  - QR factorization (Householder method)

4.2 Interpolation Error Analysis

The accuracy of the interpolating polynomial is evaluated using two complementary error measures: the absolute error and the root mean square (RMS) error.

Let $P(x)$ denote the approximating polynomial and let the reference function be

$$f(x) = \sin(5x).$$

Then the pointwise absolute error is defined as

$$E(x) = |P(x) - \sin(5x)|.$$

To assess the overall quality of the approximation across the full interval, a cumulative RMS error is computed at the sampled points $x_0, x_1, \ldots, x_i$ as

$$E_{\text{RMS}}(x_i) = \sqrt{\frac{1}{i+1} \sum_{k=0}^{i} (P(x_k) - \sin(5x_k))^2}.$$

The absolute error provides a pointwise measure of deviation and highlights local inaccuracies such as oscillations and boundary effects. In contrast, the RMS error provides a global measure of approximation quality by averaging the squared errors over the domain. While absolute error is useful for identifying localized instabilities, RMS error is more appropriate for comparing the overall performance of different approximation methods.

The interpolation error is compared for the Newton divided difference polynomial and the Chebyshev polynomial approximation. The error curves are evaluated at 600 equally spaced points in the interval $[-1, 1]$, and both the absolute error and cumulative RMS error are plotted on a logarithmic scale. The maximum error and RMS error are also summarized for direct comparison.

4.3 Observations of the plots using our Data Set

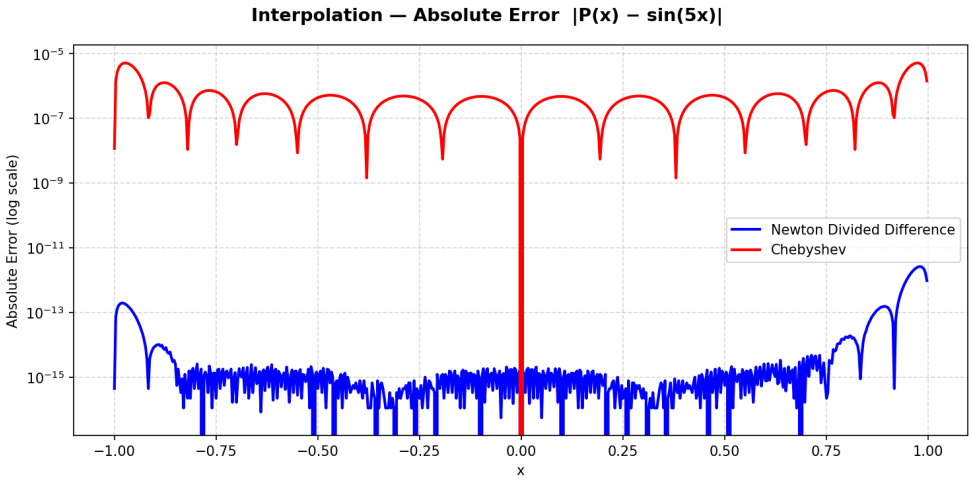

Figure 5: Absolute error $|P(x) - \sin(5x)|$ for Newton divided difference and Chebyshev approximations.

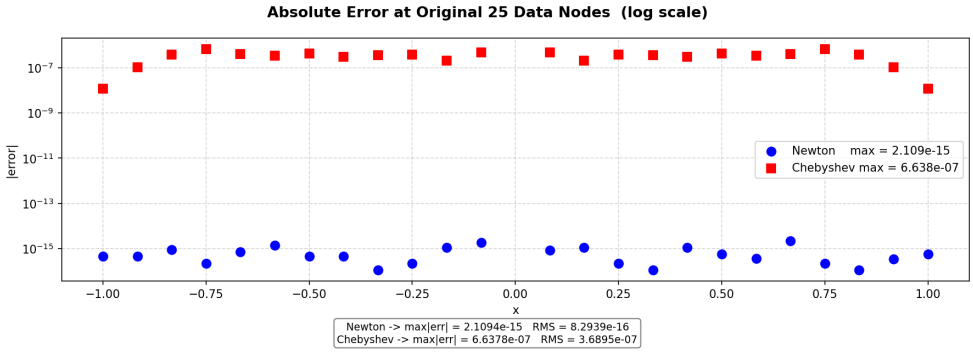

Figure 6: Absolute error $|P(x_i) - \sin(5x_i)|$ for Newton divided difference and Chebyshev approximations at the given 25 points.

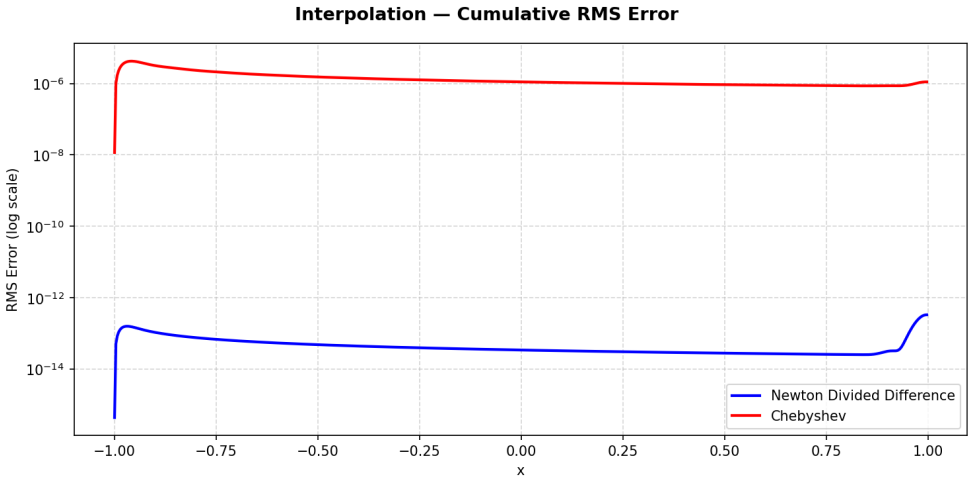

Figure 7: Cumulative RMS error analysis over the interval $[-1, 1]$.

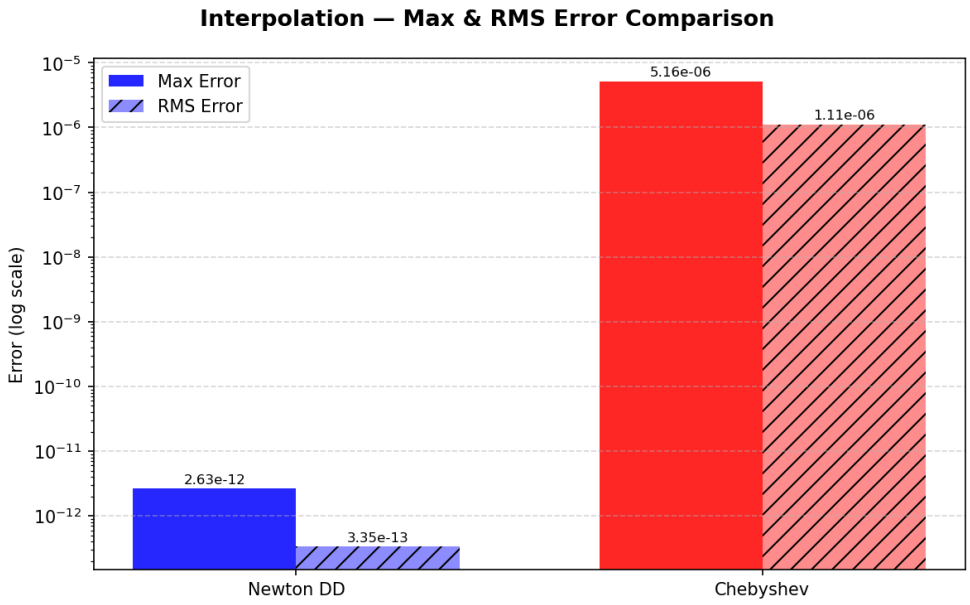

Figure 8: Comparison of maximum error and final RMS error for Newton divided difference and Chebyshev approximations.

Table 1: Maximum and RMS interpolation errors

Method	Maximum Error	RMS Error
Newton Divided Difference	2.63e-12	3.35e-13
Chebyshev Approximation	5.16e-06	1.11e-06

1. Accuracy and Polynomial Degree

- **Newton Method (Degree 24):** The plot shows the Newton interpolant reaching an absolute error floor of approximately 10^{-15}. This level of accuracy is characteristic of a high-degree polynomial ($n = 24$) matching a smooth function like $\sin(5x)$.
- **Chebyshev Method (Degree 12):** The error for the Chebyshev approximation stays higher, around 10^{-4} to 10^{-5}. This is the mathematically expected result when using a lower-degree polynomial (Degree 12) compared to the higher-degree Newton interpolant.
- **Nodal Distribution and Residuals:** For the Newton interpolant, the error at the nodes is effectively zero , confirming that the curve passes directly through every data point. In contrast, the Chebyshev approximation plot highlights how the method follows the function's general trend instead of forcing zero error at specific points.

2. Error Distribution and "Spikes"

- **Sharp Spikes:** These downward spikes occur at the specific x-coordinates where the polynomial exactly matches the target function, resulting in an absolute error of zero. Because the y-axis is logarithmic, an error of zero creates a mathematical singularity , producing the sharp vertical dips visible at each interpolation node.
- **Uniform Error Distribution:** The red Chebyshev curve follows a consistent ripple trend, maintaining a nearly constant peak error across the entire interval $[-1, 1]$. By effectively spreading the error evenly, the method avoids the large spikes typically found at the boundaries. This "averaging out" of the error ensures that the accuracy remains predictable throughout the domain.

3. Boundary Behavior (Runge's Phenomenon)

- **Newton Edge Sensitivity:** While the Newton method is extremely accurate in the center of the domain (reaching 10^{-15}), the error noticeably increases at the boundaries ($x = \pm 1$), rising to nearly 10^{-12}. This upward curve at the edges is a manifestation of **Runge's Phenomenon**, which is what we have talked about, when performing high-degree interpolation on equidistant nodes.
- **Chebyshev Stability:** In contrast, the Chebyshev error does not experience a significant increase at the edges. This demonstrates the superior stability of the Chebyshev basis in providing a consistent fit across the entire domain $[-1, 1]$.

4.4 Observations of the plots using other Data Set

Group - 1 Data Set

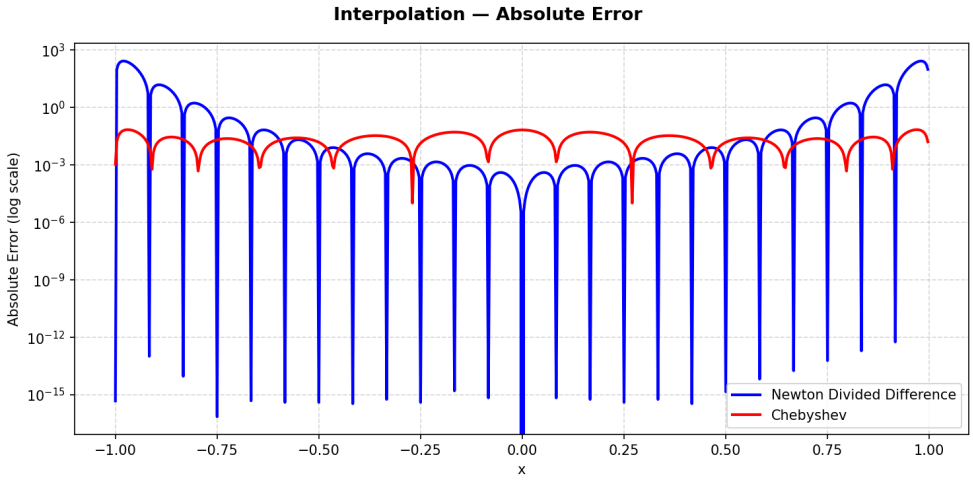

Figure 9: Absolute error $\left| P(x) - \dfrac{1}{1+25x^2} \right|$ for Newton divided difference and Chebyshev approximations.

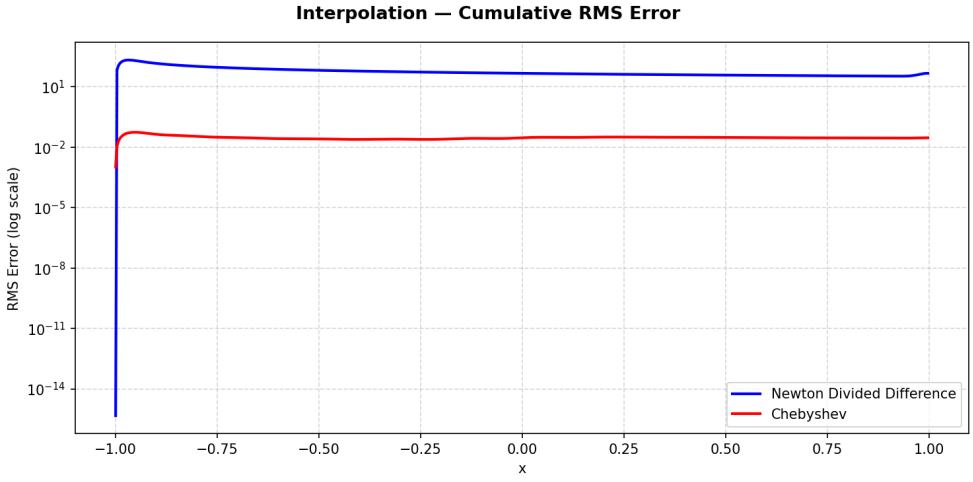

Figure 10: Cumulative RMS error analysis over the interval $[-1, 1]$.

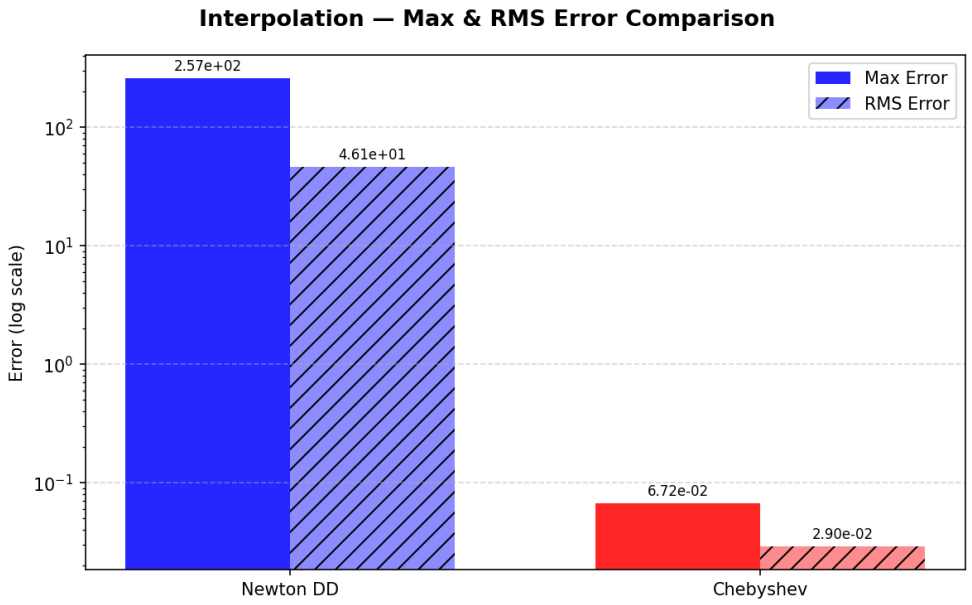

Figure 11: Comparison of maximum error and final RMS error for Newton divided difference and Chebyshev approximations.

1. Boundary Accuracy and Stability

- **Newton's Edge Problem:** The plot shows that the Newton method is extremely accurate in the middle (reaching 10^{-19}), but the error shoots up at the very ends of the graph (reaching 10^2). This happens due to to **Runge's Phenomenon**, which we talked about earlier.
- **Chebyshev's Steady Line:** The Chebyshev error line is almost perfectly flat across the whole graph. Its error (10^{-5}) at endpoint is significantly smaller (10^{-3})) and maintains this for the entire range.

3. Error Distribution

- **The Newton Gap:** There is a big gap between the "Average" (RMS) error and the "Maximum" error for the Newton method. This is because the huge errors at the edges (the worst-case) are much bigger than the accuracy in the middle.
- **Chebyshev Balance:** For Chebyshev, the Average and Maximum error lines are almost the same. This proves that the method successfully averages out the error.

Summary

This data set shows that Chebyshev is the better choice for this polynomial approximation. While Newton is very precise in the center, it becomes unreliable at the boundaries due to Runge's Phenomenon. Chebyshev in combination least squares approximation overcomes this giving us a consistent and stable fit from start to finish.

Custom Data Set ($tanh(10x)$)

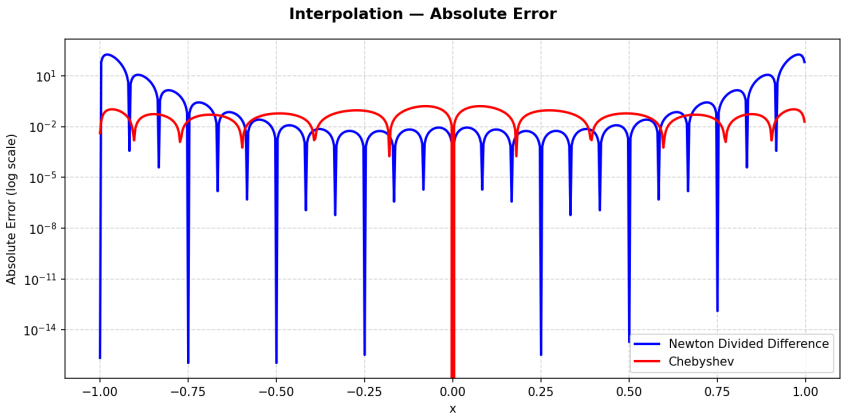

Figure 12: Absolute error $|P(x) - tanh(10x)|$ for Newton divided difference and Chebyshev approximations.

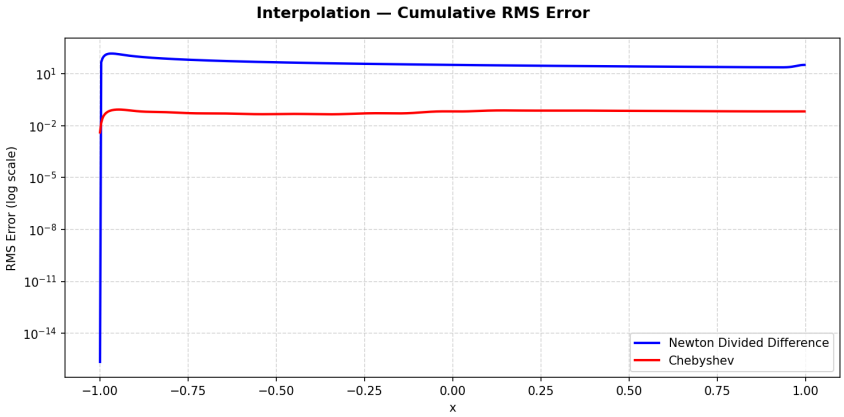

Figure 13: Cumulative RMS error analysis over the interval $[-1, 1]$.

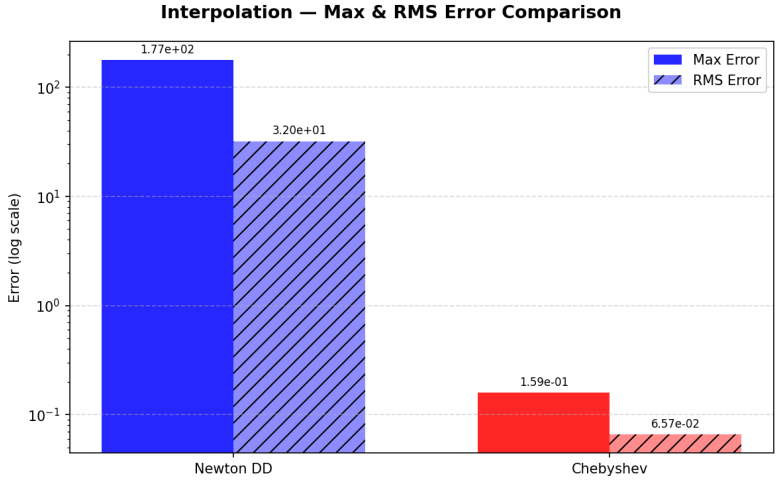

Figure 14: Comparison of maximum error and final RMS error for Newton divided difference and Chebyshev approximations.

Final Interpretation

The results for this dataset remain consistent with our previous findings. This confirms that, in general, utilizing a Chebyshev basis in a least-squares framework provides a superior polynomial approximation. This approach effectively suppresses Runge's Phenomenon and maintains a uniform error trend across the entire domain.

The uniform error trend is also reflected in the RMS plot, where the Chebyshev approximation shows a smoother and more stable trend, indicating better overall performance.

The consistency of these plots across different datasets reinforces the conclusion that a Chebyshev-based least-squares approach is numerically superior for global approximation. By distributing residuals evenly, this method successfully mitigates the boundary oscillations characteristic of Runge's Phenomenon, ensuring a stable and predictable error profile.

Therefore, while the Newton divided difference approximation may be preferable when high accuracy is required at specific points, the Chebyshev approximation is more reliable when uniform accuracy across the entire interval is desired. This highlights the importance of using both absolute error and RMS error together.

4.5 Degree vs RMS Error for Chebyshev Approximation

Our Data Set

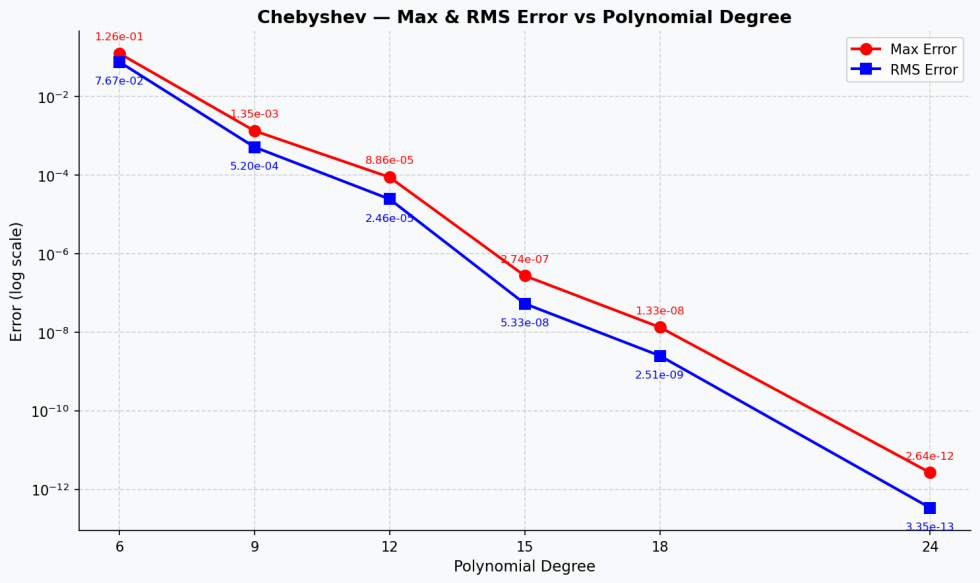

Figure 15: Cumulative RMS error analysis over the interval $[-1, 1]$.

This plot shows continuous decrease in error as the polynomial degree increases. In this case, the underlying function is smooth enough that the Chebyshev basis can accurately resolve its features without introducing numerical noise. This results in a steady improvement in both the Maximum and RMS error metrics, eventually reaching a high-precision floor. This trend confirms that for a well-conditioned system, increasing the degree allows the model to capture finer details of the dataset with increasing reliability.

This is in agreement with what we have seen earlier. For our Data Set we have seen that Chebyshev approximation has a numerically higher error than Newton Divided Difference and we know that, as we tend to n=24, Chebyshev approximation becomes similar to Newton Divided Difference.

Group - 1 Data Set

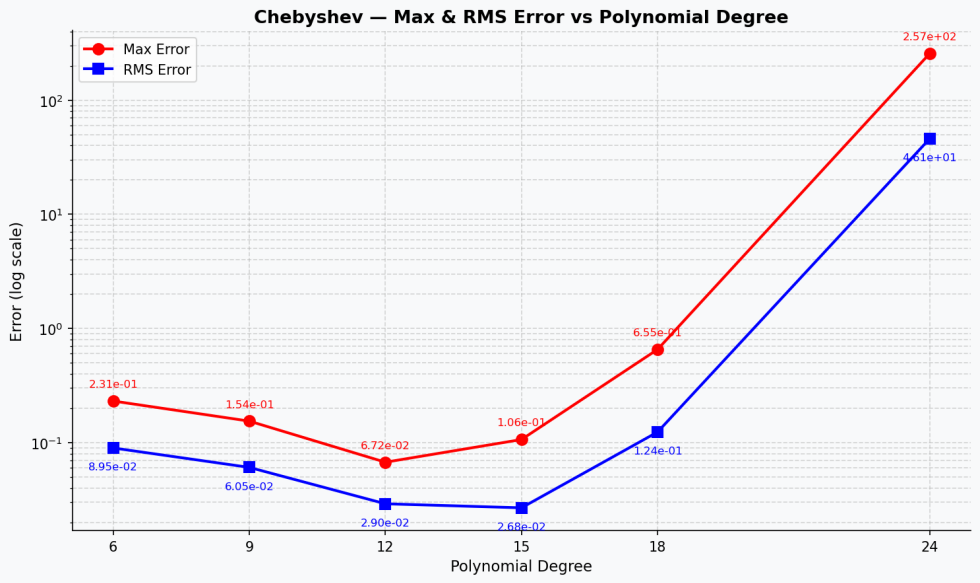

Figure 16: Cumulative RMS error analysis over the interval $[-1, 1]$.

In contrast to the earlier plot, this exhibits a classic "U-shaped" error curve, where accuracy improves until degree 12 but then fails, reaching an error of 2.57×10^2 by degree 24. This explosion occurs because the system becomes increasingly ill-conditioned as the polynomial degree n approaches the number of available data points N.

When the degree of the polynomial approaches 24, Chebyshev polynomial becomes similar to the Newton Divided Difference polynomial. As we have seen earlier with this Data Set, Runge's Phenomenon was predominant and hence the error is so high when Chebyshev approximation tends to Newton Divided Difference

4.6 Gradient Accuracy

The derivative of the interpolating polynomial is also examined, since derivative accuracy is important for root-finding methods such as Newton–Raphson. If $P(x)$ is the approximating polynomial, then the gradient error is defined as

$$E_{\text{grad}}(x) = |P'(x) - 5\cos(5x)|\,.$$

Here, $5\cos(5x)$ is the exact derivative of $\sin(5x)$. The derivative polynomial is obtained by differentiating the coefficient form of the interpolant, and the resulting error is evaluated over the same interval $[-1, 1]$ on a logarithmic scale.

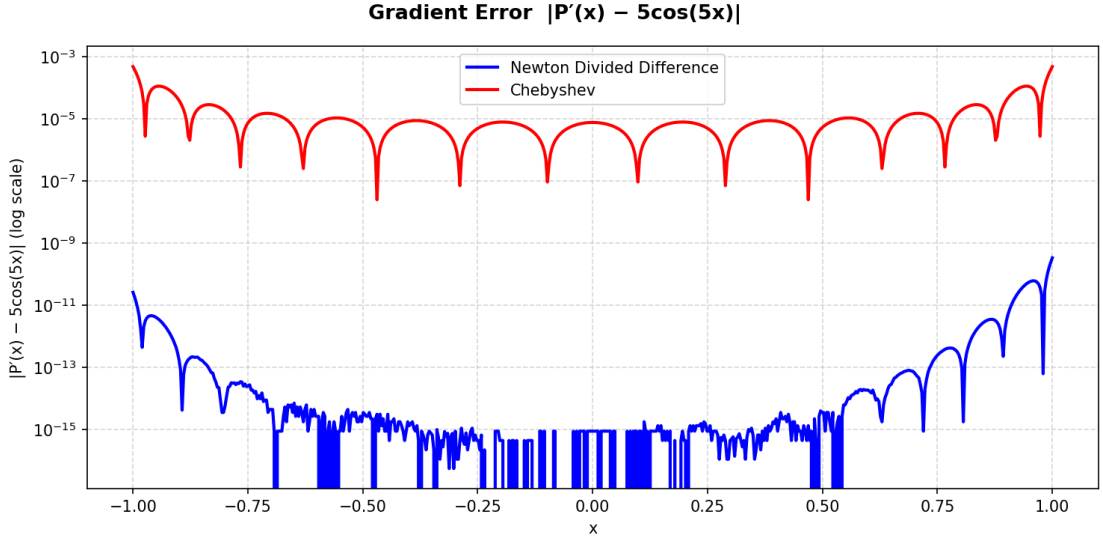

Figure 17: Gradient error $|P'(x) - 5\cos(5x)|$ for Newton divided difference and Chebyshev approximations.

Observation

The gradient error behaves similarly to the interpolation error. The Newton divided difference method shows larger variation near the endpoints, while the Chebyshev approximation remains more stable across the interval.

This is expected because differentiation amplifies small oscillations in the underlying polynomial. Therefore, the smoother behavior of the Chebyshev approximation becomes more visible in the gradient error plot.

4.7 Convergence Analysis

After obtaining the root sequences from the Secant method and the Newton–Raphson method, the convergence behavior is studied using successive errors and estimated convergence order. This analysis is carried out separately for the Newton divided difference polynomial and the Chebyshev polynomial approximation. The iteration data are exported to CSV files, and the convergence quantities are computed from the stored root estimates.

Let $\{x_n\}$ be the sequence of iterates generated by an iterative method. The successive error is defined by

$$e_n = |x_{n+1} - x_n|.$$

This quantity gives a practical measure of convergence without requiring the exact root.

Theoretical Convergence

Newton–Raphson Method:

Let α be a simple root of $g(x) = 0$, where $g(x) = P'(x)$ and $g'(\alpha) \neq 0$. The iteration is

$$x_{n+1} = x_n - \frac{g(x_n)}{g'(x_n)}.$$

Let $e_n = x_n - \alpha$. Using Taylor expansion about α,

$$g(x_n) = g(\alpha) + e_n g'(\alpha) + \frac{e_n^2}{2} g''(\alpha) + O(e_n^3).$$

Since $g(\alpha) = 0$,

$$g(x_n) = e_n g'(\alpha) + \frac{e_n^2}{2} g''(\alpha) + O(e_n^3).$$

Similarly,

$$g'(x_n) = g'(\alpha) + e_n g''(\alpha) + O(e_n^2).$$

Substituting into the iteration formula,

$$e_{n+1} = \frac{g''(\alpha)}{2g'(\alpha)} e_n^2 + O(e_n^3).$$

Hence,

$$e_{n+1} = O(e_n^2),$$

which shows **quadratic convergence**.

Secant Method:

The Secant iteration is

$$x_{n+1} = x_n - g(x_n) \frac{x_n - x_{n-1}}{g(x_n) - g(x_{n-1})}.$$

Let $e_n = x_n - \alpha$ and $e_{n-1} = x_{n-1} - \alpha$.
Using Taylor expansions,

$$g(x_n) = g'(\alpha) e_n + \frac{g''(\alpha)}{2} e_n^2 + O(e_n^3),$$

$$g(x_{n-1}) = g'(\alpha) e_{n-1} + \frac{g''(\alpha)}{2} e_{n-1}^2 + O(e_{n-1}^3).$$

Substituting and simplifying,

$$e_{n+1} \approx C\, e_n e_{n-1}.$$

Assuming convergence of order p,

$$e_{n+1} \approx K e_n^p,$$

which leads to

$$p^2 = p + 1.$$

Thus,

$$p = \frac{1 + \sqrt{5}}{2} \approx 1.618,$$

showing **superlinear convergence**.

To estimate the convergence trend, the following convergence indicator is computed:

$$p_n = \frac{\log(e_{n+1})}{\log(e_n)}.$$

In the implementation, values close to machine precision are excluded to avoid numerical instability in the logarithm. When the error becomes too small, the convergence quantity is set to zero and omitted from the plot.

The comparison is made for four cases:

- Secant method applied to the Newton divided difference polynomial
- Newton–Raphson method applied to the Newton divided difference polynomial
- Secant method applied to the Chebyshev polynomial
- Newton–Raphson method applied to the Chebyshev polynomial

The convergence results are shown in Fig. 18. The left column contains the successive error plots on a logarithmic scale, and the right column shows the estimated convergence order. The horizontal reference lines indicate the theoretical orders of Newton–Raphson and the Secant method.

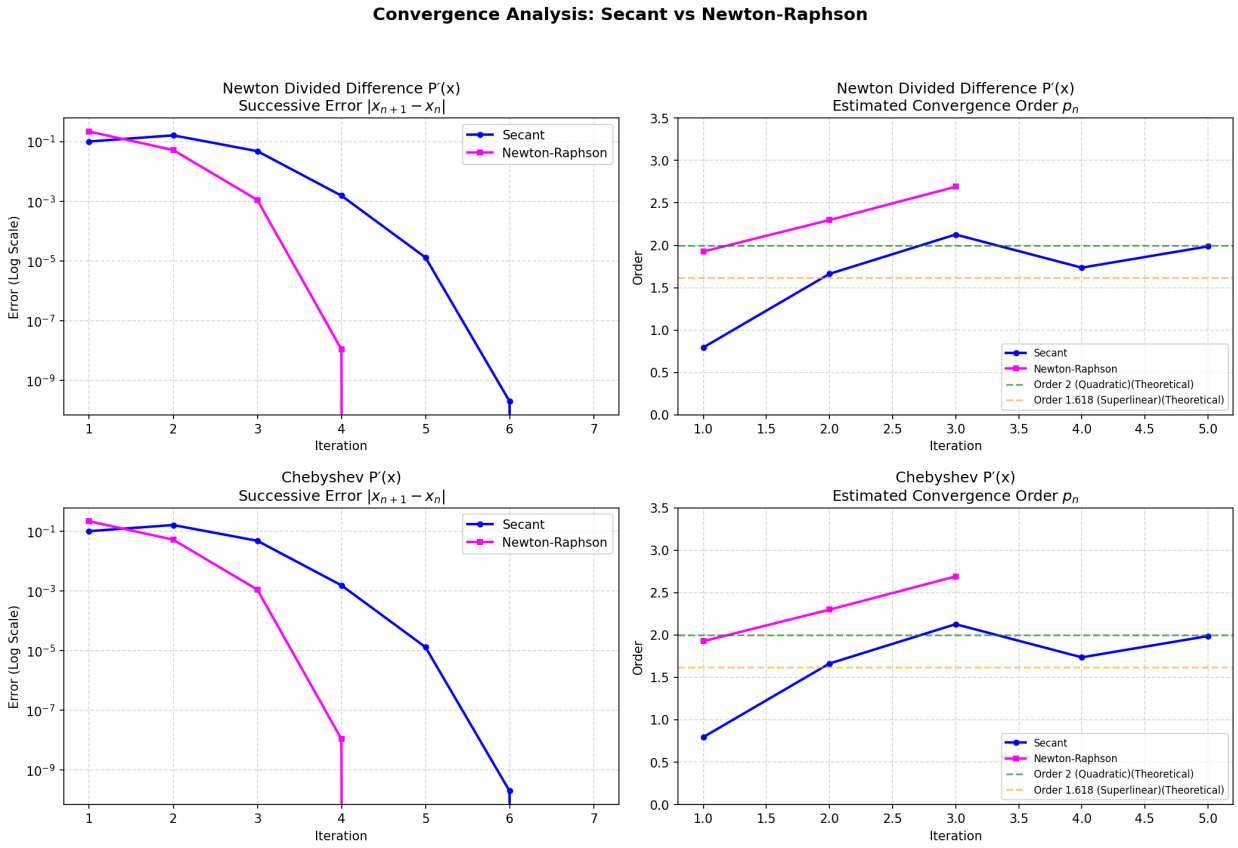

Figure 18: Convergence analysis of Secant and Newton–Raphson methods for Newton divided difference and Chebyshev approximations. Left: successive error $|x_{n+1} - x_n|$ on a logarithmic scale. Right: estimated convergence order with theoretical reference lines.

SECANT - Newton DD P′(x)

Iter	Root Estimate	Succ. Error	Conv. Order
1	0.100000000000	—	—
2	0.200000000000	1.000000e-01	0.795354
3	0.360193873300	1.601939e-01	1.663103
4	0.312633625600	4.756025e-02	2.126495
5	0.314172367400	1.538742e-03	1.735849
6	0.314159265200	1.310220e-05	1.986413
7	0.314159265400	2.000000e-10	—
8	0.314159265400	0.000000e+00	—

SECANT - Chebyshev P′(x)

Iter	Root Estimate	Succ. Error	Conv. Order
1	0.100000000000	—	—
2	0.200000000000	1.000000e-01	0.795354
3	0.360193873300	1.601939e-01	1.663103
4	0.312633625600	4.756025e-02	2.126495
5	0.314172367400	1.538742e-03	1.735849
6	0.314159265200	1.310220e-05	1.986413
7	0.314159265400	2.000000e-10	—
8	0.314159265400	0.000000e+00	—

NEWTON-RAPHSON - Newton DD P′(x)

Iter	Root Estimate	Succ. Error	Conv. Order
1	0.150000000000	—	—
2	0.364685229700	2.146852e-01	1.926235
3	0.313056210700	5.162902e-02	2.297712
4	0.314159276500	1.103066e-03	2.689755
5	0.314159265400	1.110000e-08	—
6	0.314159265400	0.000000e+00	—

NEWTON-RAPHSON - Chebyshev P′(x)

Iter	Root Estimate	Succ. Error	Conv. Order
1	0.150000000000	—	—
2	0.364685229700	2.146852e-01	1.926235
3	0.313056210700	5.162902e-02	2.297712
4	0.314159276500	1.103066e-03	2.689755
5	0.314159265400	1.110000e-08	—
6	0.314159265400	0.000000e+00	—

Observation

The Newton–Raphson method converges faster than the Secant method, as shown by the steeper decay in successive error. The estimated convergence order approaches the expected theoretical behavior: quadratic convergence for Newton–Raphson and superlinear convergence for the Secant method.

The Chebyshev-based approximation gives a more stable convergence pattern than the Newton divided difference polynomial. This is consistent with the smoother derivative behavior of the Chebyshev approximation and its better numerical conditioning.

Table 2: Expected convergence behavior

Method	Theoretical Order	Observed Trend
Secant Method	≈ 1.618	Superlinear
Newton–Raphson Method	2	Quadratic

True Roots of 5cos(5x)	Newton Polynomial Roots	Chebyshev Polynomial Roots
−0.9424777960769379	−0.9424777960768031	−0.9424823149026370
−0.3141592653589793	−0.3141592653589793	−0.3141591286956197
0.3141592653589793	0.3141592653589793	0.3141591286956197
0.9424777960769379	0.9424777960750381	0.9424823149026374

4.8 Conclusion

From the error plots, it is clear that the Newton divided difference polynomial can achieve very small local error, but its behavior is less uniform near the boundaries. The Chebyshev approximation, on the other hand, provides a more balanced error distribution across the interval.

From the gradient plots, the same pattern appears more strongly because differentiation magnifies local oscillations. This makes the advantage of the Chebyshev approximation even more visible.

From the convergence plots, Newton–Raphson consistently converges faster than the Secant method. However, the quality of the underlying polynomial also matters: the smoother Chebyshev approximation leads to more stable convergence than the Newton divided difference polynomial.

The error and convergence analysis shows that the choice of approximation method strongly influences both accuracy and stability. The Newton divided difference method is effective for interpolation, but the Chebyshev approximation gives better uniform behavior, especially when error distribution across the full interval is important.

For root finding, Newton–Raphson is faster than the Secant method, but its performance depends on the quality of the derivative information. Overall, the Chebyshev approximation combined with Newton–Raphson provides the most stable and reliable numerical behavior in this project.